

Team Notebook

just like TT

October 4, 2022

Contents

1 Contest	2	3.3 MFMC	10	5 Misc	19
1.1 Build Systems	2	3.4 articulation points	10	5.1 Submask Enumeration	19
1.2 template	2	3.5 bellman ford	11	5.2 bits	19
1.3 tester	2	3.6 binary lifting	11	5.3 digit dp	19
2 Data Structures	2	3.7 bipartite matching	13	5.4 faster hash table	21
2.1 Sqrt Decomposition	2	3.8 bridges	13	5.5 lis	21
2.2 bit 2d	3	3.9 dijkstra	14	5.6 rng	21
2.3 bit	3	3.10 eulerian cycle	14	6 Notes	21
2.4 dsu	3	3.11 eulerian path	15	6.1 DSU without Path Compression	21
2.5 merge sort tree	4	3.12 flow	15	6.2 BIT TRICKS	21
2.6 modified queue	4	3.13 floyd warshall	16	6.3 MATH	21
2.7 ordered set	5	3.14 ham cycle	16	6.4 GRAPHS	21
2.8 segment tree 2d	5	3.15 ham path	16	6.5 CMP	22
2.9 segment tree	6	3.16 kruskal	16	6.6 TWO POINTERS	22
2.10 sparse table 2d	8	3.17 scc	17	7 Strings	22
2.11 sparse table	8	3.18 topological sorting	18	7.1 manachers algorithm	22
3 Graphs	9	4 Math	18	7.2 prefix function	23
3.1 0-1 bfs	9	4.1 Euler's totient function	18	7.3 string hashing	23
3.2 2 SAT	9	4.2 Extended Euclidean Algorithm	18	7.4 suffix array	23
		4.3 factor	18	7.5 trie	24
		4.4 mint	19	7.6 z function	25

1 Contest

1.1 Build Systems

```
{
// fast
"cmd":["ulimit -s unlimited && g++ -std=c++17 -D UBC=1 -D
LOCAL=1 -Wshadow -Wall $file_name -o $file_base_name -
O2 -Wno-unused-result && timeout 5s ./$file_base_name"
],
// safe
"cmd":["ulimit -s unlimited && g++ -std=c++17 -D UBC=1 -D
LOCAL=1 -Wshadow -Wall $file_name -o $file_base_name -g
-fsanitize=address -fsanitize=undefined -
D_GLIBCXX_DEBUG && timeout 5s ./$file_base_name"],
// no limit
"cmd":["ulimit -s unlimited && g++ -std=c++17 -D UBC=1 -D
LOCAL=1 -Wshadow -Wall $file_name -o $file_base_name -
O2 -Wno-unused-result && ./$file_base_name"],
"quiet": false ,
"shell": true ,
"working_dir": "${file_path}",
"selector": "source.c++"
}
```

1.2 template

```
// generator
#ifdef LOCAL
freopen("input.txt", "w", stdout);
#endif

// fast i/o
ios::sync_with_stdio(false);
cin.tie(0);

// file
#ifdef LOCAL
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
#endif

// optimizations
#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,popcnt,lzcnt")
```

```
// precision
cout << fixed << setprecision(15);
```

1.3 tester

```
g++ temp.cpp -static -DONLINE_JUDGE -lm -s -x -fno-stack-
protector -O2 -std=c++17 -D__USE_MINGW_ANSI_STDIO=0 -o
temp
g++ brute_force.cpp -static -DONLINE_JUDGE -lm -s -x -fno-
stack-protector -O2 -std=c++17 -D__USE_MINGW_ANSI_STDIO
=0 -o brute_force
g++ generator.cpp -static -DONLINE_JUDGE -lm -s -x -fno-
stack-protector -O2 -std=c++17 -D__USE_MINGW_ANSI_STDIO
=0 -o generator
```

```
> input.txt
> output.txt
> failed_cases.txt
> output_brute.txt
```

```
for ((i = 1; ;i++)) do
printf "[TEST #i]"
./generator > input.txt
./brute_force < input.txt > output_brute.txt
./temp < input.txt > output.txt
```

```
if diff -w output_brute.txt output.txt
then
printf "[-OK-]\n"
else
printf "\n"
echo "[-*WA-*]"
cat input.txt >> failed_cases.txt
printf "\n"
echo "[-FOUND FAILING TEST!-]"
printf "\n"
echo "[INPUT]"
printf "\n"
cat failed_cases.txt
printf "\n"
echo "[BRUTE FORCE OUTPUT]"
printf "\n"
cat output_brute.txt
printf "\n"
echo "[TEMP OUTPUT]"
printf "\n"
cat output.txt
printf "\n"
break
```

```
fi
done
```

2 Data Structures

2.1 Sqrt Decomposition

```
void remove(idx); // TODO: remove value at idx from data
structure
void add(idx); // TODO: add value at idx from data
structure
int get_answer(); // TODO: extract the current answer of the
data structure
```

```
int block_size;
```

```
struct Query {
int l, r, idx;
bool operator<(Query other) const
{
return make_pair(l / block_size, r) <
make_pair(other.l / block_size, other.r);
}
};
```

```
vector<int> mo_s_algorithm(vector<Query> queries) {
vector<int> answers(queries.size());
sort(queries.begin(), queries.end());
```

```
// TODO: initialize data structure
```

```
int cur_l = 0;
int cur_r = -1;
// invariant: data structure will always reflect the
range [cur_l, cur_r]
for (Query q : queries) {
while (cur_l > q.l) {
cur_l--;
add(cur_l);
}
while (cur_r < q.r) {
cur_r++;
add(cur_r);
}
while (cur_l < q.l) {
remove(cur_l);
cur_l++;
}
}
```

```

        while (cur_r > q.r) {
            remove(cur_r);
            cur_r--;
        }
        answers[q.idx] = get_answer();
    }
    return answers;
}

```

2.2 bit 2d

```

struct BIT
{
    ll N;
    vector<ll> T;

    BIT(ll n)
    {
        N = n;
        T.assign(N + 1, 0);
    }

    void add(ll x, ll val)
    {
        while (x <= N)
        {
            T[x] += val;
            x += (x & -x);
        }
    }

    ll psum(ll x)
    {
        ll res = 0;
        while (x > 0)
        {
            res += T[x];
            x -= (x & -x);
        }
        return res;
    }

    ll rsum(ll a, ll b)
    {
        return psum(b) - psum(a - 1);
    }
};

struct BIT2D

```

```

{
    ll N, M;
    vector<BIT> T;

    BIT2D(ll n, ll m)
    {
        N = n;
        M = m;
        T.assign(N + 1, BIT(M));
    }

    void add(ll x, ll y, ll val)
    {
        while (x <= N)
        {
            T[x].add(y, val);
            x += (x & -x);
        }
    }

    ll psum(ll x, ll y)
    {
        ll res = 0;
        while (x > 0)
        {
            res += T[x].psum(y);
            x -= (x & -x);
        }
        return res;
    }

    ll rsum(ll x1, ll y1, ll x2, ll y2)
    {
        return psum(x2, y2) - psum(x1 - 1, y2) - psum(x2, y1 - 1)
            + psum(x1 - 1, y1 - 1);
    }
};

```

2.3 bit

```

struct BIT
{
    ll N;
    vector<ll> T;

    BIT(ll n)
    {
        N = n;
        T.assign(N + 1, 0);
    }
};

```

```

}

ll pref(ll x)
{
    ll res = 0;
    while (x > 0)
    {
        res += T[x];
        x -= (x & -x);
    }
    return res;
}

void add(ll x, ll val)
{
    while (x <= N)
    {
        T[x] += val;
        x += (x & -x);
    }
}

ll rsum(ll a, ll b)
{
    return pref(b) - pref(a - 1);
}
};

```

2.4 dsu

```

struct UnionFind
{
    ll N, cc;
    vector<ll> parent, rank;

    UnionFind(ll n)
    {
        N = n;
        cc = N;
        rank.assign(N, 1);
        parent.assign(N, 0);
        iota(parent.begin(), parent.end(), 0);
    }

    ll find(ll x)
    {
        if (x == parent[x])
            return x;
        else

```

```

    return parent[x] = find(parent[x]);
}

void merge(ll x, ll y)
{
    x = find(x);
    y = find(y);
    if (x != y)
    {
        if (rank[x] < rank[y])
            swap(x, y);
        parent[y] = x;
        rank[x] += rank[y];
        cc--;
    }
}
};

```

2.5 merge sort tree

// we basically keep sorted vector at each node
 // to find number of elements $\leq x$ in a range, we find it
 for all $\log(n)$ nodes and add up
 // Note: for updates use ordered set instead of vector

```

struct SegTree
{
    SegTree *lChild, *rChild;
    ll l, r, len;
    vector<ll> v;

    SegTree(ll il, ll ir, vector<ll>& iv)
    {
        l = il, r = ir;
        len = r - l + 1;
        v.resize(len);
        if (l == r)
        {
            v[0] = iv[l];
            return;
        }
        ll mid = (l + r) / 2;
        lChild = new SegTree(l, mid, iv);
        rChild = new SegTree(mid + 1, r, iv);
        merge(lChild->v.begin(), lChild->v.end(), rChild->v.begin(),
            rChild->v.end(), v.begin());
    }
}

```

```

// returns number of elements lesser-equal to x in [ql, qr]
ll rangeLessEqualx(ll ql, ll qr, ll x)
{
    if (ql > r || qr < l)
        return 0ll;
    if (ql <= l && qr >= r)
    {
        auto it = upper_bound(v.begin(), v.end(), x);
        return ll(it - v.begin());
    }
    return lChild->rangeLessEqualx(ql, qr, x) + rChild->
        rangeLessEqualx(ql, qr, x);
}

~SegTree()
{
    if (l != r)
    {
        delete lChild;
        delete rChild;
    }
}

void solve()
{
    ll n, Q; cin >> n >> Q;
    vector<ll> v(n);

    for (ll i = 0; i < n; i++)
        cin >> v[i];

    SegTree st(0, n - 1, v);

    for (ll q = 0; q < Q; q++)
    {
        ll L, R, k;
        cin >> L >> R >> k;
        L--; R--;

        // kth smallest element in [L, R]
        // binary search on ans
        ll l = 0, r = 1e10, ans = 1e10;
        while (l <= r)
        {
            ll mid = (l + r) / 2;
            if (st.rangeLessEqualx(L, R, mid) < k)
            {
                ans = mid;
            }
        }
    }
}

```

```

    l = mid + 1;
}
else
    r = mid - 1;
}

cout << ans + 1 << "\n";
}
}

```

2.6 modified queue

```

// usage:
// auto fun = [&](ll x, ll y) { return min(x, y); };
// ModifiedQueue<ll, decltype(fun)> mq(fun);

template <typename T, class F = function<T(const T&, const T
    &>>>
struct ModifiedQueue
{
    stack<pair<T, T>> s1, s2;
    F func;

    ModifiedQueue(const F& f) : func(f) {}

    ModifiedQueue() {}
    // usage -- for vector of mq's
    // vector < ModifiedQueue<ll> > v1(m);
    // v1[i] = ModifiedQueue<ll>(fun);

    void push(T x) // O(1)
    {
        T mval;
        if(s1.empty())
            mval = x;
        else
            mval = func(x, s1.top().ss);
        s1.push({x, mval});
    }

    T pop() // amortized O(1)
    {
        assert(!(s1.empty() && s2.empty()));
        if(s2.empty())
        {
            while(!s1.empty())
            {
                T temp = s1.top().ff;
                s1.pop();
            }
        }
    }
}

```

```

    T mval;
    if(s2.empty())
        mval = temp;
    else
        mval = func(temp, s2.top().ss);
    s2.push({temp, mval});
}
}
T removed = s2.top().ff;
s2.pop();
return removed;
}

T front() // amortized O(1)
{
    assert(!(s1.empty() && s2.empty()));
    if(s2.empty())
    {
        while(!s1.empty())
        {
            T temp = s1.top().ff;
            s1.pop();
            T mval;
            if(s2.empty())
                mval = temp;
            else
                mval = func(temp, s2.top().ss);
            s2.push({temp, mval});
        }
    }
    return s2.top().ff;
}

T get() // O(1)
{
    assert(!(s1.empty() && s2.empty()));
    if(s1.empty())
        return s2.top().ss;
    else if(s2.empty())
        return s1.top().ss;
    else
        return func(s1.top().ss, s2.top().ss);
}

void clear()
{
    while(!s1.empty())
        s1.pop();
    while(!s2.empty())
        s2.pop();
}

```

```

}

bool empty()
{
    return (s1.empty() && s2.empty());
}
};

```

2.7 ordered set

```

#include <bits/extc++.h>
using namespace __gnu_pbds;

template<class key, class cmp = less<key>>
using ordered_set = tree<key, null_type, cmp, rb_tree_tag,
tree_order_statistics_node_update>;

// order_of_key(X) : Returns number of items strictly
//                  smaller than X
// find_by_order(k) : Returns itr of k-th element in a set (
//                  counting from zero)

```

2.8 segment tree 2d

```

// https://cses.fi/problemset/task/1739/
// this gives TLE due to O(C * log(n) * log(m)) update time
// optimal solution for invertible operations is BIT2D as it
// don't have const. C before log

struct SegTree
{
    SegTree *lChild, *rChild;
    ll l, r, val;

    SegTree(ll il, ll ir)
    {
        l = il, r = ir;
        if (l == r)
        {
            lChild = rChild = nullptr;
            val = 0;
            return;
        }
        ll mid = (l + r) / 2;
        lChild = new SegTree(l, mid);
        rChild = new SegTree(mid + 1, r);
        recalc();
    }
}

```

```

}

SegTree(ll il, ll ir, vector<ll>& v)
{
    l = il, r = ir;
    if (l == r)
    {
        lChild = rChild = nullptr;
        val = v[l];
        return;
    }
    ll mid = (l + r) / 2;
    lChild = new SegTree(l, mid, v);
    rChild = new SegTree(mid + 1, r, v);
    recalc();
}

void recalc()
{
    if (l == r)
        return;
    else
        val = lChild->val + rChild->val;
}

ll rangeSum(ll ql, ll qr)
{
    if (ql > r || qr < l)
        return 0ll;
    if (ql <= l && qr >= r)
        return val;
    return lChild->rangeSum(ql, qr) + rChild->rangeSum(ql, qr);
}

void pointUpdate(ll index, ll newVal)
{
    if (l == r)
    {
        val = newVal;
        return;
    }
    if (index <= lChild->r)
        lChild->pointUpdate(index, newVal);
    else
        rChild->pointUpdate(index, newVal);
    recalc();
}

~SegTree()

```

```

    {
        delete lChild;
        delete rChild;
    }
};

struct SegTree2D
{
    SegTree2D *lChild, *rChild;
    ll l, r;
    SegTree *d;

    SegTree2D(ll il, ll ir, vector<vector<ll>>& v, ll m)
    {
        l = il, r = ir;
        if (l == r)
        {
            lChild = rChild = nullptr;
            d = new SegTree(0, m - 1, v[l]);
            return;
        }
        ll mid = (l + r) / 2;
        lChild = new SegTree2D(l, mid, v, m);
        rChild = new SegTree2D(mid + 1, r, v, m);
        d = new SegTree(0, m - 1);
        merge(d, lChild->d, rChild->d);
    }

    void merge(SegTree *res, SegTree *a, SegTree *b)
    {
        if (!res)
            return;
        res->val = a->val + b->val;
        merge(res->lChild, a->lChild, b->lChild);
        merge(res->rChild, a->rChild, b->rChild);
    }

    ll rangeSum(ll x1, ll x2, ll y1, ll y2)
    {
        if (x1 > r || x2 < l)
            return 0ll;
        if (x1 <= l && x2 >= r)
            return d->rangeSum(y1, y2);
        return lChild->rangeSum(x1, x2, y1, y2) + rChild->
            rangeSum(x1, x2, y1, y2);
    }

    void pointMerge(SegTree *res, SegTree *a, SegTree *b, ll
        index)
    {

```

```

        if (!res)
            return;
        res->val = a->val + b->val;
        if ((res->lChild) && (index <= res->lChild->r))
            pointMerge(res->lChild, a->lChild, b->lChild, index);
        else
            pointMerge(res->rChild, a->rChild, b->rChild, index);
    }

    void pointUpdate(ll x, ll y, ll newVal)
    {
        if (l == r)
        {
            d->pointUpdate(y, newVal);
            return;
        }
        if (x <= lChild->r)
            lChild->pointUpdate(x, y, newVal);
        else
            rChild->pointUpdate(x, y, newVal);
        pointMerge(d, lChild->d, rChild->d, y);
    }

    ~SegTree2D()
    {
        delete lChild;
        delete rChild;
        delete d;
    }
};

void solve()
{
    ll n, q; cin >> n >> q;
    vector<vector<ll>> v(n, vector<ll>(n));
    for (ll i = 0; i < n; i++)
    {
        string s; cin >> s;
        for (ll j = 0; j < n; j++)
            v[i][j] = (s[j] == '*');
    }

    SegTree2D st(0, n - 1, v, n);

    for (ll i = 0; i < q; i++)
    {
        ll t; cin >> t;
        if (t == 1)
        {
            ll x, y; cin >> x >> y;

```

```

            x--; y--;
            v[x][y] = !v[x][y];
            st.pointUpdate(x, y, v[x][y]);
        }
        else
        {
            ll x1, y1, x2, y2;
            cin >> x1 >> y1 >> x2 >> y2;
            x1--; y1--; x2--; y2--;
            cout << st.rangeSum(x1, x2, y1, y2) << "\n";
        }
    }
}

```

2.9 segment tree

// version: point update range query [sum]

```

struct SegTree
{
    SegTree *lChild, *rChild;
    ll val, l, r;

    SegTree(ll il, ll ir, vector<ll>& v)
    {
        l = il, r = ir;
        if (l == r)
        {
            val = v[l];
            return;
        }
        ll mid = (l + r) / 2;
        lChild = new SegTree(l, mid, v);
        rChild = new SegTree(mid + 1, r, v);
        recalc();
    }

    void recalc()
    {
        if (l == r)
            return;
        else
            val = lChild->val + rChild->val;
    }

    ll rangeSum(ll ql, ll qr)
    {
        if (ql > r || qr < l)
            return 0ll;

```

```

    if (ql <= l && qr >= r)
        return val;
    return lChild->rangeSum(ql, qr) + rChild->rangeSum(ql, qr);
}

void pointUpdate(ll index, ll newVal)
{
    if (l == r)
    {
        val = newVal;
        return;
    }
    if (index <= lChild->r)
        lChild->pointUpdate(index, newVal);
    else
        rChild->pointUpdate(index, newVal);
    recalc();
}

~SegTree()
{
    if (l != r)
    {
        delete lChild;
        delete rChild;
    }
}

};

// version: range update range query [sum]

struct SegTree
{
    SegTree *lChild, *rChild;
    ll val, lazy, l, r, len;

    SegTree(ll il, ll ir, vector<ll>& v)
    {
        l = il, r = ir, lazy = 0;
        len = (r - l + 1);
        if (l == r)
        {
            val = v[l];
            return;
        }
        ll mid = (l + r) / 2;

```

```

        lChild = new SegTree(l, mid, v);
        rChild = new SegTree(mid + 1, r, v);
        recalc();
    }

    void recalc()
    {
        if (l == r)
            return;
        else
            val = lChild->val + rChild->val;
    }

    void propagate() // correct this node and push lazy to childs
    {
        if (lazy != 0)
        {
            val += lazy * len;
            if (l != r)
            {
                lChild->lazy += lazy;
                rChild->lazy += lazy;
            }
            lazy = 0;
        }
    }

    void rangeUpdate(ll ql, ll qr, ll inc)
    {
        propagate();
        if (ql > r || qr < l)
            return;
        if (ql <= l && qr >= r)
        {
            lazy += inc;
            propagate();
            return;
        }
        lChild->rangeUpdate(ql, qr, inc);
        rChild->rangeUpdate(ql, qr, inc);
        recalc();
    }

    ll rangeSum(ll ql, ll qr)
    {
        propagate();
        if (ql > r || qr < l)
            return 0ll;
        if (ql <= l && qr >= r)

```

```

        return val;
        return lChild->rangeSum(ql, qr) + rChild->rangeSum(ql, qr);
    }

    ~SegTree()
    {
        if (l != r)
        {
            delete lChild;
            delete rChild;
        }
    }
};

// version: using node for more parameters
// https://www.spoj.com/problems/GSS1/

struct node
{
    ll psum;
    ll ssum;
    ll tsum;
    ll mss;

    // Note: make sure that merge(node, default) == node
    node(ll a = -INF, ll b = -INF, ll c = 0, ll d = -INF)
    {
        psum = a;
        ssum = b;
        tsum = c;
        mss = d;
    }
};

struct SegTree
{
    SegTree *lChild, *rChild;
    ll l, r;
    node d;

    SegTree(ll il, ll ir, vector<ll>& v)
    {
        l = il, r = ir;
        if (l == r)
        {
            d = node(v[l], v[l], v[l], v[l]);

```

```

    return;
}
ll mid = (l + r) / 2;
lChild = new SegTree(l, mid, v);
rChild = new SegTree(mid + 1, r, v);
d = merge(lChild->d, rChild->d);
}

node merge(node a, node b)
{
    node res;
    res.psum = max(a.psum, a.tsum + b.psum);
    res.ssum = max(b.ssum, b.tsum + a.ssum);
    res.tsum = a.tsum + b.tsum;
    res.mss = max({a.mss, b.mss, a.ssum + b.psum});
    return res;
}

node rangeMss(ll ql, ll qr)
{
    if (ql > r || qr < l)
        return node();
    if (ql <= l && qr >= r)
        return d;
    return merge(lChild->rangeMss(ql, qr), rChild->rangeMss(
        ql, qr));
}

void pointUpdate(ll index, ll newVal)
{
    if (l == r)
    {
        d = node(newVal, newVal, newVal, newVal);
        return;
    }

    if (index <= lChild->r)
        lChild->pointUpdate(index, newVal);
    else
        rChild->pointUpdate(index, newVal);

    d = merge(lChild->d, rChild->d);
}

~SegTree()
{
    if (l != r)
    {
        delete lChild;
        delete rChild;
    }
}

```

```

    }
}
};

void solve()
{
    ll n, m; cin >> n;
    vector<ll> v(n);

    for (ll i = 0; i < n; i++)
        cin >> v[i];

    SegTree st(0, n - 1, v);

    cin >> m;

    for (ll i = 0; i < m; i++)
    {
        ll x, y; cin >> x >> y;
        x--; y--;
        cout << st.rangeMss(x, y).mss << "\n";
    }
}

```

2.10 sparse table 2d

```

const ll Mx = 1010;
const ll LGMx = 64 - __builtin_clzll(Mx);
int table[LGMx][LGMx][Mx][Mx];

// usage -- idempotent function
// auto fun = [&](int x, int y) { return min(x, y); };
// SparseTable2D<int, decltype(fun)> st(arr2D, fun);

template <typename T, class F = function<T(const T&, const T
    &>>>
struct SparseTable2D
{
    ll N, M;
    F func;

    SparseTable2D(const vector<vector<T>>& a, const F& f) :
        func(f) // O(NMLogNLogM)
    {
        N = sz(a), M = sz(a[0]);
        ll lgN = 64 - __builtin_clzll(N);
        ll lgM = 64 - __builtin_clzll(M);

        // building sparse table for each row
    }
}

```

```

for(ll ir = 0; ir < N; ir++)
{
    for(ll ic = 0; ic < M; ic++)
        table[0][0][ir][ic] = a[ir][ic];

    for(ll jc = 1; jc < lgM; jc++)
    {
        for(ll ic = 0; ic <= M - (1 << jc); ic++)
            table[0][jc][ir][ic] = func(table[0][jc - 1][ir][ic]
                , table[0][jc - 1][ir][ic + (1 << (jc - 1))]);
    }
}

// building the entire table
for(ll jr = 1; jr < lgN; jr++)
    for(ll ir = 0; ir < N; ir++)
        for(ll jc = 0; jc < lgM; jc++)
            for(ll ic = 0; ic < M; ic++)
                table[jr][jc][ir][ic] = func(table[jr-1][jc][ir]
                    , table[jr-1][jc][ir + (1 << (jr - 1))
                        ][ic]);
}

T get(ll x1, ll y1, ll x2, ll y2) const // O(1)
{
    assert(0 <= x1 && 0 <= y1 && x1 <= x2 && y1 <= y2 && x2
        <= N - 1 && y2 <= M - 1);
    ll lgx = 64 - __builtin_clzll(x2 - x1 + 1) - 1;
    ll lgy = 64 - __builtin_clzll(y2 - y1 + 1) - 1;
    T R1 = func(table[lgx][lgy][x1][y1], table[lgx][lgy][x1][
        y2 - (1 << lgy) + 1]);
    T R2 = func(table[lgx][lgy][x2 - (1 << lgx) + 1][y1],
        table[lgx][lgy][x2 - (1 << lgx) + 1][y2 - (1 << lgy)
            + 1]);
    return func(R1, R2);
}
};

```

2.11 sparse table

```

// const ll Mx = 600100;

// ll LG[Mx];

// void pre_log()
// {
//     LG[1] = 0;
//     for(ll i = 2; i < Mx; i++)

```

```
// LG[i] = LG[i >> 1] + 1;
// }

// usage -- idempotent function
// auto fun = [&](ll x, ll y) { return min(x, y); };
// SparseTable<ll, decltype(fun)> st(arr, fun);
template <typename T, class F = function<T(const T&, const T
&>>>
struct SparseTable
{
    ll N;
    vector<vector<T>>> mat;
    F func;

    SparseTable() {}
    // usage -- for vector of spt's
    // vector < SparseTable<ll> > v1(m);
    // v1[i] = SparseTable<ll>(arr2d[i], fun);

    SparseTable(const vector<T>& a, const F& f) : func(f) // 0
        (NLogN)
    {
        N = sz(a);
        ll max_log = 64 - __builtin_clzll(N); // LG[N] + 1;
        mat.resize(max_log);
        mat[0] = a;
        for(ll j = 1; j < max_log; j++)
        {
            mat[j].resize(N - (1 << j) + 1);
            for(ll i = 0; i <= N - (1 << j); i++)
            {
                mat[j][i] = func(mat[j - 1][i], mat[j - 1][i + (1 <<
                    (j - 1))]);
            }
        }

        T get(ll from, ll to) const // 0(1)
        {
            assert(0 <= from && from <= to && to <= N - 1);
            ll lg = 64 - __builtin_clzll(to - from + 1) - 1; // LG[to
                - from + 1];
            return func(mat[lg][from], mat[lg][to - (1 << lg) + 1]);
        }
    };
};
```

3 Graphs

3.1 0-1 bfs

```
// https://www.codechef.com/problems/REVERSE
// https://www.codechef.com/viewsolution/48525003
// https://cp-algorithms.com/graph/01_bfs.html

void solve()
{
    ll n, m; cin >> n >> m;
    vector<vector<array<ll, 2>>> adj(n);

    for (ll i = 0; i < m; i++)
    {
        ll x, y; cin >> x >> y;
        x--; y--;
        adj[x].push_back({y, 0});
        adj[y].push_back({x, 1});
    }

    deque<ll> q;
    vector<ll> d(n, INF);
    q.push_back(0);
    d[0] = 0;

    while (!q.empty())
    {
        ll v = q.front();
        q.pop_front();

        for (auto edge: adj[v])
        {
            ll u = edge[0];
            ll w = edge[1];
            if (d[u] > d[v] + w)
            {
                d[u] = d[v] + w;
                if (w == 1)
                    q.push_back(u);
                else
                    q.push_front(u);
            }
        }
    }

    if (d[n - 1] == INF)
        cout << "-1" << "\n";
    else
        cout << d[n - 1] << "\n";
}
```

```
}
```

3.2 2 SAT

```
ll n,m; // n ppl each 2 wishes regarding m toppings
vll g,grev;
vll visited,completetimes,vertexid;
ll id = 1;

void fillorder(ll x){
    visited[x] = 0;
    for (ll j = 0; j < g[x].size(); j++)
        if (visited[g[x][j]] == -1)
            fillorder(g[x][j]);
    visited[x] = 1;
    completetimes.push_back(x);
}

void dfs(ll x){
    visited[x] = 0;
    vertexid[x] = id;
    for (ll j = 0; j < grev[x].size(); j++)
        if (visited[grev[x][j]] == -1)
            dfs(grev[x][j]);
    visited[x] = 1;
}

int main(){
    //time_t start,end;
    //time(&start);
    ios_base::sync_with_stdio(false), cin.tie(0), cout.tie(0)
    ;
    cin>>n>>m;
    g.resize(2*m+1);
    grev.resize(2*m+1);
    visited.resize(2*m+1);
    vertexid.resize(2*m+1);
    fill (visited.begin(), visited.end(), -1);
    // m toppings n conditions
    // 2*n edges, 2*m vertices
    // 1,2,3,...m => 1,-1,2,-2,...m,-m => 1,2,3...2*m
    for (ll i = 1; i <= n; i++){
        char x,y;
        ll a,b;
        cin>>x; cin>>a; cin>>y; cin>>b;
        if (x == '+' && y == '+'){
            g[2*a].push_back(2*b-1);
            g[2*b].push_back(2*a-1);
            grev[2*b-1].push_back(2*a);
        }
    }
}
```

```

    grev[2*a-1].push_back(2*b);
}
if (x == '-' && y == '+'){
    g[2*a-1].push_back(2*b-1);
    g[2*b].push_back(2*a);
    grev[2*b-1].push_back(2*a-1);
    grev[2*a].push_back(2*b);
}
if (x == '+' && y == '-'){
    g[2*a].push_back(2*b);
    g[2*b-1].push_back(2*a-1);
    grev[2*b].push_back(2*a);
    grev[2*a-1].push_back(2*b-1);
}
if (x == '-' && y == '-'){
    g[2*a-1].push_back(2*b);
    g[2*b-1].push_back(2*a);
    grev[2*b].push_back(2*a-1);
    grev[2*a].push_back(2*b-1);
}
}
for (ll i = 1; i <= 2*m; i++)
    if (visited[i] == -1)
        fillorder(i);
fill (visited.begin(), visited.end(), -1);
while (completetimes.size() != 0){
    ll temp = completetimes[completetimes.size()-1];
    completetimes.pop_back();
    if (visited[temp] == -1){
        dfs(temp);
        id++;
    }
}
ll flag = 0;
vector <char> ans(m+1);
for (ll i = 1; i <= m; i++){
    if (vertexid[2*i-1] == vertexid[2*i])
        flag = 1;
    else if (vertexid[2*i-1] < vertexid[2*i])
        ans[i] = '-';
    else if (vertexid[2*i-1] > vertexid[2*i])
        ans[i] = '+';
}
if (flag == 0)
    for (ll i = 1; i <= m; i++)
        cout<<ans[i]<<" ";
else
    cout<<"IMPOSSIBLE\n";

//time(&end);

```

```

//double time_taken = double(end-start);
//cout<<"\nTime taken for execution is "<< fixed <<
    time_taken << setprecision(5)<<" sec\n";
return 0;
}

```

3.3 MFMC

```

ll n,m;
ll g[505][505];
ll orig[505][505];
ll source, sink;
vector <bool> visited(505, false);
ll dfs(ll i, ll amount){
    if (i == sink)
        return amount;
    visited[i] = true;

    for (ll j = 1; j <= n; j++){
        if (g[i][j] > 0 && visited[j] == false){
            int sent = dfs(j, min(amount, g[i][j]));
            if (sent > 0) {
                g[i][j] -= sent;
                g[j][i] += sent;
                return sent;
            }
        }
    }
    return 0;
}

ll run(){
    ll total = 0;
    ll sent = -1;
    while (sent != 0) {
        visited.assign(n,false);
        sent = dfs(source, INT_MAX);
        total += sent;
    }
    return total;
}

void dfs2(int i){
    visited[i] = true;
    for (ll j = 1; j <= n; j++){
        if (g[i][j] > 0 && visited[j] == false){
            dfs2(j);
        }
    }
}

```

```

}

void solve(){
    cin>>n>>m;
    source = 1, sink = n;
    visited.assign(n,false);
    for (ll i = 1; i <= n; i++)
        for (ll j = 0; j <= n; j++)
            g[i][j] = 0, orig[i][j] = 0;
    for (ll i = 1; i <= m; i++){
        ll a,b;
        cin>>a>>b;
        g[a][b] += 1;
        g[b][a] += 1;
        orig[a][b] += 1;
        orig[b][a] += 1;
    }
    ll flow = run();
    cout<<flow<<"\n";
    visited.assign(n,false);
    dfs2(1);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (visited[i] == true && visited[j] == false &&
                orig[i][j] > 0)
                cout << i << " " << j << "\n";
}

```

3.4 articulation points

```

// https://www.spoj.com/problems/SUBMERGE/
// https://cp-algorithms.com/graph/cutpoints.html

// Note:
// A vertex can be an articulation point even if there
// exists a back-edge going over it
// i.e if it has multiple children and only some children's
// subtrees have back-edges that go over it while others
// don't
// => A vertex is not an articulation point if and only if
// an edge passes over it from "each of its subtrees".

// The root of the dfs tree is a special case: it is an
// articulation point if and only if it has more than one
// child in the dfs tree.

void solve()
{

```

```

ll n, m; cin >> n >> m;

if (n == 0 && m == 0)
    exit(0);

vector<vector<ll>> adj(n);

for (ll i = 0; i < m; i++)
{
    ll x, y; cin >> x >> y;
    x--; y--;
    adj[x].push_back(y);
    adj[y].push_back(x);
}

ll timer = 1;
vector<ll> vis(n, 0), tin(n), isAp(n, 0), low(n);

function<void(ll, ll)> dfs = [&](ll v, ll p)
{
    vis[v] = 1;
    low[v] = tin[v] = timer++;
    ll children = 0;

    for (auto x: adj[v])
    {
        if (x != p)
        {
            if (!vis[x])
            {
                dfs(x, v);
                low[v] = min(low[v], low[x]);
                if (low[x] >= tin[v] && p != -1)
                    isAp[v] = 1;
                children++;
            }
            else
                low[v] = min(low[v], tin[x]);
        }
    }

    if (p == -1 && children > 1)
        isAp[v] = 1;
};

for (ll i = 0; i < n; i++)
{
    if (!vis[i])
        dfs(i, -1);
}

```

3.5 bellman ford

```

// https://cses.fi/problemset/task/1197/
// https://cp-algorithms.com/graph/bellman_ford.html
// https://cp-algorithms.com/graph/finding-negative-cycle-in-graph.html

void solve()
{
    ll n, m; cin >> n >> m;
    vector<array<ll, 3>> edges;

    for (ll i = 0; i < m; i++)
    {
        ll x, y, w;
        cin >> x >> y >> w;
        x--; y--;
        edges.push_back({x, y, w});
    }

    vector<ll> d(n, 0), p(n, -1);

    // Note: if we start bellman ford only from 0 we can only
    // find if any negative cycle reachable from 0
    // so here d[v] is the minimum distance from any vertex to
    // v and it will converge in n - 1 iterations
    // if it doesn't have a negative cycle.

    ll X = -1; // last changed vertex

    for (ll i = 0; i < n; i++)
    {
        X = -1; // initialized after every iteration

        for (auto e: edges)
        {
            if (d[e[0]] + e[2] < d[e[1]])
            {
                d[e[1]] = max(-INF, d[e[0]] + e[2]);
                p[e[1]] = e[0];
                X = p[e[1]];
            }
        }
    }
}

```

```

if (X == -1) // no change in the nth iteration
    cout << "NO" << "\n";
else
{
    cout << "YES" << "\n";

    // vertex X will either lie in a negative weight cycle,
    // or is reachable from it
    for (ll i = 0; i < n; i++)
        X = p[X];

    // now X will be in the cycle

    vector<ll> path;

    for (ll i = X; ; i = p[i])
    {
        path.push_back(i);
        if (i == X && path.size() > 1) // completed rotation
            break;
    }

    reverse(path.begin(), path.end());

    for (auto x: path)
        cout << x + 1 << " ";
    cout << "\n";
}
}

```

3.6 binary lifting

```

// version: kth ancestor
// Problem: https://cses.fi/problemset/task/1687/

using ll = long long;
const ll MX = 20;

void solve()
{
    ll n, q; cin >> n >> q;
    vector<vector<ll>> adj(n);

    for (ll i = 1; i < n; i++)
    {
        ll p; cin >> p; p--;
        adj[i].push_back(p);
        adj[p].push_back(i);
    }
}

```

```

vector<vector<ll>> up(n, vector<ll>(MX + 1));
vector<ll> depth(n, 0);

function<void(ll, ll)> dfs = [&](ll node, ll parent)
{
    up[node][0] = parent;
    for (ll i = 1; i <= MX; i++)
        up[node][i] = up[up[node][i - 1]][i - 1];

    for (auto x: adj[node])
    {
        if (x != parent)
        {
            depth[x] = depth[node] + 1;
            dfs(x, node);
        }
    }
};

ll root = 0;
dfs(root, root);

auto ancestor = [&](ll x, ll d)
{
    for (ll i = 0; i <= MX; i++)
    {
        if (d & (1ll << i))
            x = up[x][i];
    }
    return x;
};

for (ll i = 0; i < q; i++)
{
    ll v, k; cin >> v >> k;
    v--;
    if (k > depth[v])
        cout << -1 << "\n";
    else
        cout << ancestor(v, k) + 1 << "\n";
}

// version: LCA
// https://cses.fi/problemset/task/1688/

```

```

using ll = long long;
const ll MX = 20;

void solve()
{
    ll n, q; cin >> n >> q;
    vector<vector<ll>> adj(n);

    for (ll i = 1; i < n; i++)
    {
        ll p; cin >> p; p--;
        adj[i].push_back(p);
        adj[p].push_back(i);
    }

    vector<vector<ll>> up(n, vector<ll>(MX + 1));
    vector<ll> depth(n, 0);

    function<void(ll, ll)> dfs = [&](ll node, ll parent)
    {
        up[node][0] = parent;
        for (ll i = 1; i <= MX; i++)
            up[node][i] = up[up[node][i - 1]][i - 1];

        for (auto x: adj[node])
        {
            if (x != parent)
            {
                depth[x] = depth[node] + 1;
                dfs(x, node);
            }
        }
    };

    ll root = 0;
    dfs(root, root);

    auto ancestor = [&](ll x, ll d)
    {
        for (ll i = 0; i <= MX; i++)
        {
            if (d & (1ll << i))
                x = up[x][i];
        }
        return x;
    };

    auto lca = [&](ll u, ll v)
    {
        if (depth[u] < depth[v])

```

```

        swap(u, v);

        u = ancestor(u, depth[u] - depth[v]);

        if (u == v)
            return u;

        for (ll i = MX; i >= 0; i--)
        {
            if (up[u][i] != up[v][i])
            {
                u = up[u][i];
                v = up[v][i];
            }
        }

        return up[u][0];
    };

    for (ll i = 0; i < q; i++)
    {
        ll a, b; cin >> a >> b;
        a--; b--;
        cout << lca(a, b) + 1 << "\n";
    }

    // version: Path Min (path aggregates)
    // https://codeforces.com/gym/102694/problem/D

    using ll = long long;

    const ll INF = 1e18;
    const ll MX = 20;

    void solve()
    {
        ll n, edges; cin >> n >> edges;
        vector<vector<array<ll, 2>>> adj(n);

        for (ll i = 0; i < edges; i++)
        {
            ll x, y; cin >> x >> y;
            ll val; cin >> val;
            x--; y--;
            adj[x].push_back({y, val});
            adj[y].push_back({x, val});

```

```

}

vector<vector<ll>> up(n, vector<ll>(MX + 1));
vector<vector<ll>> upMin(n, vector<ll>(MX + 1));
vector<ll> depth(n, 0);

function<void(ll, ll, ll)> dfs = [&](ll node, ll parent,
    ll val)
{
    up[node][0] = parent;
    upMin[node][0] = val;
    for (ll i = 1; i <= MX; i++)
    {
        up[node][i] = up[up[node][i - 1]][i - 1];
        upMin[node][i] = min(upMin[node][i - 1], upMin[up[node]
            ][i - 1][i - 1]);
    }

    for (auto x: adj[node])
    {
        if (x[0] != parent)
        {
            depth[x[0]] = depth[node] + 1;
            dfs(x[0], node, x[1]);
        }
    }
};

ll root = 0;
dfs(root, root, INF);

auto ancestor = [&](ll x, ll d)
{
    for (ll i = 0; i <= MX; i++)
    {
        if (d & (1ll << i))
            x = up[x][i];
    }
    return x;
};

auto lca = [&](ll u, ll v)
{
    if (depth[u] < depth[v])
        swap(u, v);

    u = ancestor(u, depth[u] - depth[v]);

    if (u == v)
        return u;
}

```

```

for (ll i = MX; i >= 0; i--)
{
    if (up[u][i] != up[v][i])
    {
        u = up[u][i];
        v = up[v][i];
    }
}

return up[u][0];
};

auto pathMin = [&](ll u, ll v)
{
    ll m = lca(u, v), res = INF;
    ll um = depth[u] - depth[m];
    ll vm = depth[v] - depth[m];

    for (ll i = 0; i <= MX; i++)
    {
        if (um & (1ll << i))
        {
            res = min(res, upMin[u][i]);
            u = up[u][i];
        }
    }

    for (ll i = 0; i <= MX; i++)
    {
        if (vm & (1ll << i))
        {
            res = min(res, upMin[v][i]);
            v = up[v][i];
        }
    }

    return res;
};

ll q; cin >> q;
for (ll i = 0; i < q; i++)
{
    ll a, b; cin >> a >> b;
    a--; b--;
    ll ans = pathMin(a, b);
    if (ans == INF)
        cout << 0 << "\n";
    else
        cout << ans << "\n";
}

```

```

}
}

```

3.7 bipartite matching

```

vll g; vll mt; vector<bool> visited; ll cnt = 0;
bool try_kuhn(ll i){
    if (visited[i])
        return false;
    visited[i] = true;
    for (int to : g[i]) {
        if (mt[to] == -1 || try_kuhn(mt[to])) {
            mt[to] = i;
            return true;
        }
    }
    return false;
}

void solve(){
    ll n,m,k;
    cin>>n>>m>>k;
    g.resize(n+1); mt.assign(m+1,-1);
    for (ll i = 0; i < k; i++){
        ll a,b;
        cin>>a>>b;
        g[a].push_back(b);
    }
    for (ll i = 1; i <= n; i++){
        visited.assign(n+1,false);
        try_kuhn(i);
    }
    for (ll i = 1; i <= m; i++)
        if (mt[i] != -1)
            cnt++;
    printf("%lld\n",cnt);
    for (ll i = 1; i <= m; i++)
        if (mt[i] != -1)
            printf("%lld %lld\n", mt[i], i);
}

```

3.8 bridges

```

// https://www.spoj.com/problems/EC_P/
// https://cp-algorithms.com/graph/bridge-searching.html
// https://codeforces.com/blog/entry/68138

```

```
// Note:
// A tree-edge uv is a bridge if and only if there is no
// back-edge that "passes over" uv.
// A back-edge is never a bridge.

void solve()
{
    ll n, m; cin >> n >> m;
    vector<vector<ll>> adj(n);

    for (ll i = 0; i < m; i++)
    {
        ll x, y; cin >> x >> y;
        x--; y--;
        adj[x].push_back(y);
        adj[y].push_back(x);
    }

    ll timer = 1;
    vector<ll> tin(n), low(n), vis(n, 0);
    vector<array<ll, 2>> bridges;

    function<void(ll, ll)> dfs = [&](ll v, ll p)
    {
        vis[v] = 1;
        tin[v] = low[v] = timer++;

        for (auto x: adj[v])
        {
            if (x != p)
            {
                if (!vis[x])
                {
                    dfs(x, v);
                    low[v] = min(low[v], low[x]);
                    if (low[x] > tin[v])
                        bridges.push_back({min(x, v) + 1, max(x, v) + 1});
                }
                else
                    low[v] = min(low[v], tin[x]);
            }
        }
    };

    for (ll i = 0; i < n; i++)
    {
        if (!vis[i])
            dfs(i, -1);
    }
}
```

```
sort(bridges.begin(), bridges.end());

if (bridges.empty())
    cout << "Sin bloques" << "\n";
else
{
    cout << bridges.size() << "\n";
    for (auto p: bridges)
        cout << p[0] << " " << p[1] << "\n";
}
}
```

3.9 dijkstra

// <https://cp-algorithms.com/graph/dijkstra.html>
 // <https://cses.fi/problemset/task/1671/>

```
void solve()
{
    ll n, m; cin >> n >> m;
    vector<vector<array<ll, 2>>> adj(n);

    for (ll i = 0; i < m; i++)
    {
        ll x, y, w;
        cin >> x >> y >> w;
        x--; y--;
        adj[x].push_back({y, w});
    }

    vector<ll> d(n, INF);
    set<array<ll, 2>> q;
    q.insert({0, 0});
    d[0] = 0;

    while (!q.empty())
    {
        ll v = (*q.begin())[1];
        q.erase(q.begin());

        for (auto edge: adj[v])
        {
            ll u = edge[0];
            ll w = edge[1];
            if (d[u] > d[v] + w)
            {
                q.erase({d[u], u});
                d[u] = d[v] + w;
            }
        }
    }
}
```

```
q.insert({d[u], u});
    }
}

for (auto x: d)
    cout << x << " ";
cout << "\n";
}
```

3.10 eulerian cycle

```
void solve()
{
    ll n, m;
    vll visited(1e5+1, -1);
    vll degree(1e5+1, 0);
    stack<ll, vll> s;
    vll answer;

    void dfs(ll x) {
        visited[x] = 1;
        for (ll j = 0; j < g[x].size(); j++) {
            if (visited[g[x][j]] == -1)
                dfs(g[x][j]);
        }
    }

    int main() {
        ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

        cin >> n >> m;
        g.resize(n+1); visited.resize(n+1);
        for (ll i = 1; i <= m; i++) {
            ll a, b;
            cin >> a >> b;
            g[a].push_back(b);
            g[b].push_back(a);
            degree[a]++;
            degree[b]++;
        }

        dfs(1);
        for (ll i = 1; i <= n; i++) {
            if (visited[i] == -1 && degree[i] != 0) {
                cout << "IMPOSSIBLE\n";
                return 0;
            }
        }

        for (ll i = 1; i <= n; i++) {
            if (degree[i] % 2 == 1) {
                cout << "IMPOSSIBLE\n";
            }
        }
    }
}
```

```

        return 0;
    }
}
s.push(1);
while (!s.empty()){
    ll temp = s.top();
    if (degree[temp] == 0){
        answer.push_back(temp);
        s.pop();
    }
    else {
        degree[g[temp][0]]--;
        degree[temp]--;
        s.push(g[temp][0]);
        g[g[temp][0]].erase(find(g[g[temp][0]].begin(), g[
            temp][0].end(), temp));
        g[temp].erase(g[temp].begin());
    }
}
for (ll j = 0; j < answer.size(); j++)
    cout<<answer[j]<<" ";
cout<<"\n";
return 0;
}

```

3.11 eulerian path

```

vll g;
ll n,m;
vll visited(1e5+1,-1);
vll degree(1e5+1,0);
stack <ll,vll> s;
vll answer;
void dfs(ll x){
    visited[x] = 1;
    for (ll j = 0; j < g[x].size(); j++){
        if (visited[g[x][j]] == -1)
            dfs(g[x][j]);
    }
}
int main(){
    ios_base::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    ;
    cin>>n>>m;
    g.resize(n+1); visited.resize(n+1);
    for (ll i = 1; i <= m; i++){
        ll a,b;
        cin>>a>>b;
    }
}

```

```

    g[a].push_back(b);
    //g[b].push_back(a);
    degree[a]++;
    degree[b]++;
}

dfs(1);
for (ll i = 1; i <= n; i++){
    if (visited[i] == -1 && degree[i] != 0){
        cout<<"IMPOSSIBLE\n";
        return 0;
    }
}
for (ll i = 2; i < n; i++){
    if (degree[i]%2 == 1){
        cout<<"IMPOSSIBLE\n";
        return 0;
    }
}
if (degree[1]%2 == 1 && degree[n]%2 == 1){
    g[n].push_back(1);
    //g[1].push_back(n);
    degree[1]++; degree[n]++;
}
else {
    cout<<"IMPOSSIBLE\n";
    return 0;
}

s.push(1);
while (!s.empty()){
    ll temp = s.top();
    if (degree[temp] == 0){
        answer.push_back(temp);
        s.pop();
    }
    else {
        degree[g[temp][0]]--;
        degree[temp]--;
        s.push(g[temp][0]);
        //g[g[temp][0]].erase(find(g[g[temp][0]].begin(),
            g[g[temp][0]].end(), temp));
        g[temp].erase(g[temp].begin());
    }
}
reverse(answer.begin(), answer.end());
vll newans;
for (ll j = 0; j < answer.size(); j++){
    if (answer[j] == n && answer[j+1] == 1){
        for (ll k = j+1; k < answer.size(); k++)

```

```

        newans.push_back(answer[k]);
        for (ll k = 1; k <= j; k++)
            newans.push_back(answer[k]);
        break;
    }
}
for (ll j = 0; j < newans.size(); j++)
    cout<<newans[j]<<" ";
cout<<"\n";
return 0;
}

```

3.12 flow

```

ll n,m;
ll g[505][505];
ll source, sink;
vector <bool> visited(505, false);
ll dfs(ll i, ll amount){
    if (i == sink)
        return amount;
    visited[i] = true;

    for (ll j = 1; j <= n; j++){
        if (g[i][j] > 0 && visited[j] == false){
            int sent = dfs(j, min(amount, g[i][j]));
            if (sent > 0) {
                g[i][j] -= sent;
                g[j][i] += sent;
                return sent;
            }
        }
    }
    return 0;
}

ll run(){
    ll total = 0;
    ll sent = -1;
    while (sent != 0) {
        visited.assign(n,false);
        sent = dfs(source, INT_MAX);
        total += sent;
    }
    return total;
}

void solve(){
    cin>>n>>m;
}

```

```

source = 1, sink = n;
visited.assign(n,false);
for (ll i = 1; i <= n; i++){
    for (ll j = 0; j <= n; j++){
        g[i][j] = 0;
    }
    for (ll i = 1; i <= m; i++){
        ll a,b,c;
        cin>>a>>b>>c;
        g[a][b] += c;
    }
    cout << run() << "\n";
}

```

3.13 floyd warshall

<https://cses.fi/problemset/task/1672/>

<https://cp-algorithms.com/graph/all-pair-shortest-path-floyd-warshall.html>

```

void solve()
{
    ll n, m, q;
    cin >> n >> m >> q;

    vector<vector<ll>> d(n, vector<ll>(n, INF));

    for (ll i = 0; i < m; i++)
    {
        ll x, y, w;
        cin >> x >> y >> w;
        x--; y--;
        d[x][y] = min(d[x][y], w);
        d[y][x] = min(d[y][x], w);
    }

    for (ll i = 0; i < n; i++)
        d[i][i] = 0;

    for (ll k = 0; k < n; k++)
        for (ll i = 0; i < n; i++)
            for (ll j = 0; j < n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    for (ll i = 0; i < q; i++)
    {
        ll x, y; cin >> x >> y;
        x--; y--;
        cout << ((d[x][y] == INF) ? -1 : d[x][y]) << "\n";
    }
}

```

```

}

```

3.14 ham cycle

```

vector<vector<bool>> g(1e5+1,vector<bool>(1e5+1,false));
ll n,m;
vll path(1e5,-1);
vector<bool> visited(1e5+1,false);
bool hamcycle(ll idx){
    if (idx == n)
        return g[path[n-1]][path[0]];
    for (ll i = 1; i <= n; i++){
        if (!visited[i] && g[path[idx-1]][i]){
            path[idx] = i;
            visited[i] = true;
            if (hamcycle(idx+1))
                return true;
            path[idx] = -1;
            visited[i] = false;
        }
    }
    return false;
}
int main(){
    ios_base::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    ;
    cin>>n>>m;
    for (ll i = 1; i <= m; i++){
        ll a,b;
        cin>>a>>b;
        g[a][b] = g[b][a] = true;
    }

    path.resize(n);
    path[0] = 1; // starts at post office
    visited[1] = true;
    if (hamcycle(1)){
        for (ll i = 0; i < n; i++)
            cout<<path[i]<<" ";
        cout<<"1\n";
    }
    else
        cout<<"IMPOSSIBLE\n";
    return 0;
}

```

3.15 ham path

```

vll g;
ll n,m;
int main(){ // ham path from 1 to n
    ios_base::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    ;
    cin>>n>>m;
    g.resize(n+1);
    for (ll i = 1; i <= m; i++){
        ll a,b;
        cin>>a>>b;
        g[a].push_back(b);
    }
    ll dp[n+1][1<<n]; // in mask if ith bit is set then i+1
    // is in chosen subset (... ,3,2,1,0)
    for (ll i = 0; i < n+1; i++)
        for (ll j = 0; j < (1<<n); j++)
            dp[i][j] = 0;
    dp[1][1] = 1;
    for (ll mask = 1; mask < (1<<n); mask++){
        //cout<<mask<<endl;
        if (mask>>(n-1)&1){ // stuff where n is in subset
            // if (mask != (1<<n)-1)
                continue;
        }
        for (ll j = 1; j <= n; j++){
            //cout<<"j = "<<j<<endl;
            if (mask>>(j-1) & 1){ // stuff where j is in
                subset
                for (ll k = 0; k < g[j].size(); k++){
                    //cout<<"seeing vertex "<<g[j][k]<<endl;
                    if (mask>>(g[j][k]-1) & 1)
                        continue;
                    dp[g[j][k]][mask ^ (1<<(g[j][k]-1))] = (dp
                        [j][mask] + dp[g[j][k]][mask ^ (1<<(g
                        [j][k]-1))])%MOD;
                }
            }
        }
    }
    cout<<dp[n][(1<<n)-1]%MOD<<"\n";
    return 0;
}

```

3.16 kruskal

https://cp-algorithms.com/graph/mst_kruskal.html

```
// https://cses.fi/problemset/task/1675/

void solve()
{
    ll n, m; cin >> n >> m;
    vector<array<ll, 3>> edges;

    for (ll i = 0; i < m; i++)
    {
        ll x, y, w;
        cin >> x >> y >> w;
        x--; y--;
        edges.push_back({w, x, y});
    }

    sort(edges.begin(), edges.end());

    UnionFind uf(n);

    ll cost = 0, ecnt = 0;

    for (auto edge: edges)
    {
        if (uf.find(edge[1]) != uf.find(edge[2]))
        {
            cost += edge[0];
            ecnt++;
            uf.merge(edge[1], edge[2]);
        }
    }

    if (ecnt != n - 1)
        cout << "IMPOSSIBLE" << "\n";
    else
        cout << cost << "\n";
}
```

3.17 SCC

```
// https://cp-algorithms.com/graph/strongly-connected-
// components.html
// https://www.cs.dartmouth.edu/~deepc/Courses/S20/lects/
// lec11.pdf

// Kosaraju's two-pass algorithm

// Note:
// DFS guarantees that the vertex(scc) with the biggest
// finishing time will be in the SOURCE of condensation
```

```
graph,
// but the vertex(scc) with the smallest finishing time may
// not be in SINK of condensation graph.

// Example:
// V = {a, b, c, d}
// E = {ab, ba, ac, cd, dc}
// dfs: a-b-a-c-d-c-a
// when we start dfs at 'a', 'b' has smallest finishing time
// but it is not in the SINK
// 'a' has the largest finishing time it is in the SOURCE of
// G_SCC

// Any edge (C, C') in condensation graph comes from
// a component with a larger value of tout to component with
// a smaller value.

// the resulting algorithm's scheme generates strongly
// connected components by decreasing order of their exit
// times,
// thus it generates components - vertices of condensation
// graph - in topological sort order.

// https://cses.fi/problemset/task/1683/

void solve()
{
    ll n, m; cin >> n >> m;
    vector<vector<ll>> adj(n);
    vector<vector<ll>> radj(n);

    for (ll i = 0; i < m; i++)
    {
        ll x, y; cin >> x >> y;
        x--; y--;
        adj[x].push_back(y);
        radj[y].push_back(x);
    }

    vector<ll> vis(n, 0);
    vector<ll> order;

    function<void(ll)> dfs1 = [&](ll v)
    {
        vis[v] = 1;
        for (auto x: adj[v])
        {
            if (!vis[x])
                dfs1(x);
        }
    }

    order.push_back(v);
};

for (ll i = 0; i < n; i++)
{
    if (!vis[i])
        dfs1(i);
}

reverse(order.begin(), order.end());
vis.assign(n, 0);
vector<ll> scc;
vector<vector<ll>> sccs;

function<void(ll)> dfs2 = [&](ll v)
{
    vis[v] = 1;
    scc.push_back(v);
    for (auto x: radj[v])
    {
        if (!vis[x])
            dfs2(x);
    }
};

for (auto v: order)
{
    if (!vis[v])
    {
        dfs2(v);
        sccs.push_back(scc);
        scc.clear();
    }
}

cout << sccs.size() << "\n";
vector<ll> color(n);
ll cnt = 1;

for (auto v: sccs)
{
    for (auto x: v)
        color[x] = cnt;
    cnt++;
}

for (auto x: color)
    cout << x << " ";
cout << "\n";
}
```

3.18 topological sorting

```
// https://www.spoj.com/problems/TOPOSORT/
// Lexicographically smallest topological sort
// Khan's Algorithm

// Note:
// A DAG G has at least one vertex with in-degree 0 and one
// vertex with out-degree 0
// So Pick all the vertices with in-degree as 0 and add them
// into a queue (priority queue if needed lexicographically
// smallest)
// while removing it from the queue reduce indegree of
// neighbours

void solve()
{
    ll n, m; cin >> n >> m;
    vector<vector<ll>> adj(n);
    vector<ll> indegree(n, 0);

    for (ll i = 0; i < m; i++)
    {
        ll x, y; cin >> x >> y;
        x--; y--;
        adj[x].push_back(y);
        indegree[y]++;
    }

    vector<ll> topo;
    priority_queue<ll, vector<ll>, greater<ll>> q;

    for (ll i = 0; i < n; i++)
    {
        if (indegree[i] == 0)
            q.push(i);
    }

    while (!q.empty())
    {
        ll v = q.top();
        q.pop();
        topo.push_back(v);

        for (auto x: adj[v])
        {
            indegree[x]--;
            if (indegree[x] == 0)
                q.push(x);
        }
    }
}
```

```
}

if ((ll) topo.size() < n)
    cout << "Sandro fails.";
else
{
    for (auto x: topo)
        cout << x + 1 << " ";
}
}
```

4 Math

4.1 Euler's totient function

```
// phi-function , counts the number of integers between 1
// and n inclusive, which are coprime to n.
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

4.2 Extended Euclidean Algorithm

```
// https://cp-algorithms.web.app/algebra/extended-euclid-
// algorithm.html
// ax + by = gcd(a, b);
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

```
}
```

4.3 factor

```
namespace factorizer
{
    const ll N = 200200;
    ll lp[N + 1];
    vector<ll> pr;

    void sieve()
    {
        for (ll i = 2; i <= N; ++i)
        {
            if (lp[i] == 0)
            {
                lp[i] = i;
                pr.push_back(i);
            }

            for (ll j = 0; j < ll(pr.size()) && pr[j] <= lp[i] && 1
                LL * i * pr[j] <= N; j++)
                lp[i * pr[j]] = pr[j];
        }
    }

    vector<array<ll, 2>> pfact(ll x)
    {
        vector<array<ll, 2>> factors;
        while (x > 1)
        {
            ll cnt = 0;
            ll p = lp[x];
            while (x % p == 0)
            {
                x /= p;
                cnt++;
            }
            factors.push_back({p, cnt});
        }
        return factors;
    }

    vector<ll> build_divisors(vector<array<ll, 2>> &factors)
    {
        vector<ll> divisors = {1};
        for (auto &p : factors)
        {
            ll size = ll(divisors.size());

```

```

    for (ll i = 0; i < size; i++)
    {
        ll cur = divisors[i];
        for (ll j = 0; j < p[1]; j++)
        {
            cur *= p[0];
            divisors.push_back(cur);
        }
    }
    sort(divisors.begin(), divisors.end());
    return divisors;
}

vector<ll> divisors(ll x)
{
    vector<array<ll, 2>> factors = pfact(x);
    return build_divisors(factors);
}
}

using namespace factorizer;

```

4.4 mint

```

const ll MOD = 1e9+7;

namespace mod_op
{
    ll madd(ll a, ll b)
    {
        return (a + b) % MOD;
    }

    ll mmul(ll a, ll b)
    {
        return ((a % MOD) * (b % MOD)) % MOD;
    }

    ll msub(ll a, ll b)
    {
        return (((a - b) % MOD) + MOD) % MOD;
    }

    ll mpow(ll a, ll b)
    {
        a %= MOD;
        ll res = 1;
        while (b > 0)

```

```

    {
        if (b & 1)
            res = mmul(res, a);
        a = mmul(a, a);
        b >>= 1;
    }
    return res;
}

ll minv(ll a)
{
    return mpow(a, MOD - 2);
}

ll mdiv(ll a, ll b)
{
    return mmul(a, minv(b));
}
}

using namespace mod_op;

const ll MXN = 1e6;

ll factorial[MXN + 1];
ll inv_factorial[MXN + 1];

void prepare_factorials()
{
    factorial[0] = 1;
    for (ll i = 1; i <= MXN; i++)
        factorial[i] = mmul(factorial[i - 1], i);

    inv_factorial[MXN] = minv(factorial[MXN]);

    for (ll i = MXN - 1; i >= 0; i--)
        inv_factorial[i] = mmul(inv_factorial[i + 1], i + 1);
}

ll ncr(ll n, ll r)
{
    if (n < 0 || r < 0 || r > n)
        return 0ll;
    else
        return mmul(factorial[n], mmul(inv_factorial[n - r],
                                         inv_factorial[r]));
}

```

5 Misc

5.1 Submask Enumeration

```

for (int s=m; s; s=(s-1)&m)
... you can use s ...
// zero will not be processed.

```

5.2 bits

```

int x = 5328; // 00000000000000000001010011010000
cout << __builtin_clz(x) << "\n"; // 19
cout << __builtin_ctz(x) << "\n"; // 4
cout << __builtin_popcount(x) << "\n"; // 5
cout << __builtin_parity(x) << "\n"; // 1

```

__builtin_clz(x): the number of zeros at the beginning of the number
 __builtin_ctz(x): the number of zeros at the end of the number
 __builtin_popcount(x): the number of ones in the number
 __builtin_parity(x): the parity (even or odd) of the number of ones

5.3 digit dp

```

// version 1.0 - number of numbers with some property in [L, R]

```

```

ll dp[10010][2][2][110];

```

```

void solve()
{
    string L = "1";
    string R;
    ll d; cin >> R >> d;

    // make both L, R of same length by appending 0's to L if needed
    L = string((ll) R.size() - (ll) L.size(), '0') + L;
    ll N = (ll) L.size();

    memset(dp, -1, sizeof(dp));

    /*
    level -> level in trie (msb -> lsb)
    */
}

```

```

bl    -> is 1 if the state is bounded by l
br    -> is 1 if the state is bounded by r
smd   -> sum of digits so far mod d

- initially we are at root (level = 0), where both l, r
  are bounded( / \ )
- each leaf node of trie represents a number

dp(level, bl, br, smd) is the number of numbers with sum
  of digits % d = 0
where we have sum smd already, with given bounds

The overlapping sub-problems can be observed after
  looking at trie (msb -> lsb)
*/

function<ll(ll, ll, ll, ll)> rec = [&](ll level, ll bl, ll
    br, ll smd)
{
    if (level == N)
    {
        if (smd == 0)
            return 1ll;
        else
            return 0ll;
    }

    if (dp[level][bl][br][smd] != -1)
        return dp[level][bl][br][smd];

    ll ans = 0;

    ll l = 0, r = 9;
    if (bl)
        l = L[level] - '0';
    if (br)
        r = R[level] - '0';

    for (ll i = l; i <= r; i++)
    {
        ll bl_i = bl, br_i = br;

        if (i != L[level] - '0')
            bl_i = 0;
        if (i != R[level] - '0')
            br_i = 0;

        ans = madd(ans, rec(level + 1, bl_i, br_i, (smd + i) %
            d));
    }
}

```

```

        return dp[level][bl][br][smd] = ans;
    };

    cout << rec(0, 1, 1, 0) << "\n";
}

// version 2.0 - number of pairs with some property in [L, R
    ]

// Problem:
// F(x) = sum of digits of number x.

// Given a number N, find the number of pairs (x,y) such
    that -
// 0    x < y    N
// F(x) < F(y)
// F(x) + F(y) is prime.

int dp[51][2][2][2][2][460][460][2];

void solve()
{
    vector<int> pcache(1010, -1);

    auto is_prime = [&](int n)
    {
        if (n <= 1)
            return 0;

        if (pcache[n] != -1)
            return pcache[n];

        int ans = 1;

        for (int i = 2; i * i <= n; i++)
        {
            if (n % i == 0)
            {
                ans = 0;
                break;
            }
        }

        return pcache[n] = ans;
    };

    memset(dp, -1, sizeof(dp));

    string L = "0";
}

```

```

string R; cin >> R;
L = string(int(R.size()) - int(L.size()), '0') + L;
int N = L.size();

// g -> 1 if x < y, 0 if x == y so far

function<int(int, int, int, int, int, int, int, int)> rec
    = [&](int level, int blx, int brx, int bly, int bry,
        int sx, int sy, int g)
{
    if (level == N)
    {
        if (is_prime(sx + sy) && (sx < sy) && (g == 1))
            return 1;
        else
            return 0;
    }

    if (dp[level][blx][brx][bly][bry][sx][sy][g] != -1)
        return dp[level][blx][brx][bly][bry][sx][sy][g];

    int lx = 0, rx = 9, ly = 0, ry = 9;
    if (blx)
        lx = L[level] - '0';
    if (brx)
        rx = R[level] - '0';
    if (bly)
        ly = L[level] - '0';
    if (bry)
        ry = R[level] - '0';

    int ans = 0;

    for (int i = lx; i <= rx; i++)
    {
        int blx_i = blx, brx_i = brx;

        if (i != L[level] - '0')
            blx_i = 0;
        if (i != R[level] - '0')
            brx_i = 0;

        for (int j = ly; j <= ry; j++)
        {
            int bly_j = bly, bry_j = bry;

            if (j != L[level] - '0')
                bly_j = 0;
            if (j != R[level] - '0')
                bry_j = 0;
        }
    }
}

```

```

int g_ij = g;

if (g == 0)
{
    if (level == N - 1)
    {
        if (i >= j)
            continue;
        else
            g_ij = 1;
    }
    else
    {
        if (i > j)
            continue;
        else if (i < j)
            g_ij = 1;
    }
}

ans = madd(ans, rec(level + 1, blx_i, brx_i, bly_j,
    bry_j, sx + i, sy + j, g_ij));
}

return dp[level][blx][brx][bly][bry][sx][sy][g] = ans;
};

cout << rec(0, 1, 1, 1, 1, 0, 0, 0) << "\n";
}

```

5.4 faster hash table

```

// Usage
// To use this data structure:

#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
gp_hash_table<int, int> table;

// Defeating Anti-Hash tests
// One weakness of hash tables is that mean people can find
hash collisions offline and blow up the complexity of
your hashmap. In my opinion, the easiest way of solving
this is below. There's no need to define your own
custom hash function.

```

```

const int RANDOM = chrono::high_resolution_clock::now().
    time_since_epoch().count();
struct chash {
    int operator()(int x) const { return x ^ RANDOM; }
};
gp_hash_table<key, int, chash> table;

```

5.5 lis

```

vector<ll> v;

for (auto x: arr)
{
    if (v.empty())
        v.push_back(x);
    else
    {
        if (x > v.back())
            v.push_back(x);
        else
        {
            auto it = lower_bound(v.begin(), v.end(), x);
            *it = x;
        }
    }
}

cout << v.size() << "\n";

```

5.6 rng

```

// Standard Mersenne twister seeded with time
mt19937_64 gen(chrono::high_resolution_clock::now().
    time_since_epoch().count());

ll random(ll a, ll b)
{
    // Produces random integer values uniformly distributed on
    the closed interval [a, b]
    uniform_int_distribution<ll> dis(a, b);
    return dis(gen);
}

```

6 Notes

6.1 DSU without Path Compression

- When we are using DSU without path compression, in scenarios like DSU with rollbacks, we can need to keep the DSU time complexity to a minimum.
- This is also achieved by a technique similar to small to large. Here, when merging two components, we set the parent of the smaller component to the larger component.
- Each time we traverse an edge to a parent, the overall subtree size at that parent is doubled. Implies $O(\log n)$ for find operation.

6.2 BIT TRICKS

- To get the value of the least significant bit that is one $V = (x \& (-x))$
- For Subarray & think about suffix subarray & and traverse from right to left since number of distinct values for $arr[L, r]$ is at most 64, number of set bits decreases monotonically for $r \geq L$ as r increases.

6.3 MATH

- $(x \bmod a) \bmod (ak) = (x \bmod (ak)) \bmod (a) = x \bmod (a)$
- If we want to determine the probability of moving from the state X to state Y over K steps - use Markov. {matrix exponentiation}
- It is better to take $\% \text{ MOD}$ of the matrix before exponentiation or multiplication to reduce the number of modulo operations.

6.4 GRAPHS

- A DAG G has at least one vertex with in-degree 0 and one vertex with out-degree 0

- In the DFS tree of an “undirected graph” **edges can only go from a descendant to an ancestor** i.e, No cross edges. The back-edges of the graph all connect a vertex with its descendant in the spanning tree. This is why DFS tree is so useful.

6.5 CMP

- SET

```
auto cmp = [&](ll a, ll b)
{
    return a > b;
};
set<ll, decltype(cmp)> s(cmp);
```

- MULTISSET

```
auto cmp = [&](ll a, ll b)
{
    return a > b;
};
multiset<ll, decltype(cmp)> s(cmp);
```

- PRIORITY QUEUE

```
auto cmp = [&](ll a, ll b)
{
    return a > b;
};
priority_queue<ll, vl, decltype(cmp)> pq(cmp);

priority_queue<ll, vl, greater<ll>> pq;
// min heap, default max heap
```

- ORDERED SET (NO fsanitize)

```
auto cmp = [&](ll a, ll b)
{
    return a > b;
};
ordered_set<ll, decltype(cmp)> s(cmp);
```

- USING STRUCT

```
struct student
{
    string s;
    ll r;
    bool operator<(const student &other) const
    {
        return r < other.r;
    }
};
```

6.6 TWO POINTERS

- Works if $l' > 1$ then $r' \geq r$

```
// template for 2 pointers
ll head = -1, tail = 0;
while (tail < n)
{
    while ((head + 1 < n) && check())
    {
        head++;
        // upd
    }
    ans = max(ans, head - tail + 1);
    if (tail < head)
    {
        tail++;
        // upd
    }
    else
    {
        tail++;
        head = tail - 1;
        // reset
    }
}
```

7 Strings

7.1 manachers algorithm

```
// https://cp-algorithms.com/string/manacher.html

// Note:
// d1[i] -> stores number of odd palindromes centered at i
// d2[i] -> stores number of even palindromes centered at (i - 1, 'i')
// (l, r) maintains borders of the rightmost found sub-palindrome (i. e. the palindrome with maximal r).
// Each iteration of the while loop will increase the right border r of the match segment.
// Since r can't be more than n - 1, this means that the inner while loop won't make more than n - 1 iterations

vector<ll> manachers_odd(string s)
{
    ll n = s.size();
    vector<ll> d1(n);

    for (ll i = 0, l = 0, r = -1; i < n; i++)
    {
        if (i <= r)
            d1[i] = min(r - i + 1, d1[l + r - i]);
        else
            d1[i] = 1;
        while (0 <= i - d1[i] && i + d1[i] < n && s[i - d1[i]] == s[i + d1[i]])
            d1[i]++;
        if (i + d1[i] - 1 > r)
        {
            l = i - d1[i] + 1;
            r = i + d1[i] - 1;
        }
    }

    return d1;
}

vector<ll> manachers_even(string s)
{
    ll n = s.size();
    vector<ll> d2(n);

    for (ll i = 0, l = 0, r = -1; i < n; i++)
    {
        if (i <= r)
            d2[i] = min(r - i + 1, d2[l + r - i + 1]);
```

```

else
    d2[i] = 0;
while (0 <= i - d2[i] - 1 && i + d2[i] < n && s[i - d2[i]
    - 1] == s[i + d2[i]])
    d2[i]++;
if (i + d2[i] - 1 > r)
{
    l = i - d2[i];
    r = i + d2[i] - 1;
}
}

return d2;
}

```

7.2 prefix function

```

// https://cp-algorithms.com/string/prefix-function.html

// Note:
// pi[i] is length of the longest proper prefix which is
// also a suffix of the substring s[0...i]
// pi[0] = 0
// The values of the prefix function can only increase by at
// most one
// Time complexity is O(n) because: pi[i] <= pi[i - 1] + 1
// - Each character can add most one to pi. and each, time
// we go back, the matching decreases by at least 1.
// - So we can deduce that the number of times the inner
// while loop runs cannot be more than O(n).

vector<ll> prefix_function(string s)
{
    ll n = s.size();
    vector<ll> pi(n);
    pi[0] = 0;

    for (ll i = 1; i < n; i++)
    {
        ll j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }

    return pi;
}

```

7.3 string hashing

```

// https://codeforces.com/blog/entry/60445
// https://cp-algorithms.com/string/string-hashing.html
// https://cp-algorithms.com/string/rabin-karp.html

// version: int (faster)

// Usage for MultiHash: (pick random p if needed to avoid
// hacks)
// StringHash sh1(n, s);
// StringHash sh2(n, s, 97, 1e9+9);

struct StringHash
{
    int N, P, M, iP;
    vector<int> pow, inv, prefix;

    StringHash() {}

    StringHash(int n, string s, int p = 89, int m = 1e9+7)
    {
        N = n, P = p, M = m;
        iP = minv(P);
        pow.resize(N);
        inv.resize(N);
        prefix.resize(N);

        inv[0] = pow[0] = 1;

        for (int i = 1; i < N; i++)
        {
            pow[i] = mmul(pow[i - 1], P);
            inv[i] = mmul(inv[i - 1], iP);
        }

        prefix[0] = (s[0] - '0' + 1);

        for (int i = 1; i < N; i++)
            prefix[i] = madd(prefix[i - 1], mmul(s[i] - '0' + 1,
                pow[i]));
    }

    int get(int l, int r)
    {
        if (l == 0)
            return prefix[r];
    }
}

```

```

else
    return mmul(msub(prefix[r], prefix[l - 1]), inv[l]);
}
// mint with M
};

// version: ll (generic)

vector<ll> bases =
{
    random(89, 1000),
    random(89, 1000),
    ...
};

vector<ll> primes =
{
    1000000007, 1000000009, 1000000021,
    1000000033, 1000000087, 1000000093,
    1000000097, 1000000103, 1000000123, 1000000181
};

template<ll K>
struct MultiHash
{
    StringHash sh[K];

    MultiHash(ll n, string s)
    {
        for (ll i = 0; i < K; i++)
            sh[i] = StringHash(n, s, bases[i], primes[i]);
    }

    vector<ll> get(ll l, ll r)
    {
        vector<ll> res(K);
        for (ll i = 0; i < K; i++)
            res[i] = sh[i].get(l, r);
        return res;
    }
};

```

7.4 suffix array

```

vector<int> sort_cyclic_shifts(string const& s) {
    int n = s.size();
    const int alphabet = 256;
    vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++)

```

```

    cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++)
        cnt[i] += cnt[i-1];
    for (int i = 0; i < n; i++)
        p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i-1]])
            classes++;
        c[p[i]] = classes - 1;
    }
    vector<int> pn(n), cn(n);
    for (int h = 0; (1 << h) < n; ++h) {
        for (int i = 0; i < n; i++) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0)
                pn[i] += n;
        }
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++)
            cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++)
            cnt[i] += cnt[i-1];
        for (int i = n-1; i >= 0; i--)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)
                ) % n]};
            pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1
                << h) % n]};
            if (cur != prev)
                ++classes;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    return p;
}

vector<int> suffix_array_construction(string s) {
    s += "$";
    vector<int> sorted_shifts = sort_cyclic_shifts(s);
    sorted_shifts.erase(sorted_shifts.begin());
    return sorted_shifts;
}

// Comparing two substrings of a string

```

```

// store all the intermediate results of the equivalence
// classes
int compare(int i, int j, int l, int k) {
    pair<int, int> a = {c[k][i], c[k][(i+1-(1 << k))%n]};
    pair<int, int> b = {c[k][j], c[k][(j+1-(1 << k))%n]};
    return a == b ? 0 : a < b ? -1 : 1;
}

// LCP of suffixes Pi and Pj is min(lcp(i), ... lcp(j - 1))
vector<int> lcp_construction(string const& s, vector<int>
    const& p) {
    int n = s.size();
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++)
        rank[p[i]] = i;

    int k = 0;
    vector<int> lcp(n-1, 0);
    for (int i = 0; i < n; i++) {
        if (rank[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = p[rank[i] + 1];
        while (i + k < n && j + k < n && s[i+k] == s[j+k])
            k++;
        lcp[rank[i]] = k;
        if (k)
            k--;
    }
    return lcp;
}

// distinct sub strings: C(n + 1, 2) - (lcp[0] + ... lcp[n -
    2])

```

7.5 trie

```

// version xor

// usage:
// trie t(32);
// Note: Insert 0 if needed

struct trie
{
    ll MX; // max number of bits (constant for all bit strings
    )
}

```

```

struct node
{
    node *child[2];
    ll cnt; // number of bit strings ending in the subtree(
        node) including this node

    node()
    {
        cnt = 0;
        for (ll i = 0; i < 2; i++)
            child[i] = NULL;
    }
};

node *root;

trie(ll m)
{
    MX = m;
    root = new node;
}

void insert(ll n) // msb to lsb
{
    node *cur = root;
    for (ll i = MX - 1; i >= 0; i--)
    {
        cur->cnt++;
        ll x = (n & (1ll << i)) ? 1 : 0;
        if (cur->child[x] == NULL)
            cur->child[x] = new node;
        cur = cur->child[x];
    }
    cur->cnt++;
}

void remove(ll n)
{
    node *cur = root;
    for (ll i = MX - 1; i >= 0; i--)
    {
        cur->cnt--;
        ll x = (n & (1ll << i)) ? 1 : 0;
        cur = cur->child[x];
    }
    cur->cnt--;
}

ll max_xor(ll n) // returns max(n ^ t_i)
{
}

```

```

assert(root->cnt > 0);

ll ans = 0;
node *cur = root;
for (ll i = MX - 1; i >= 0; i--)
{
    ll x = (n & (1ll << i)) ? 1 : 0;
    if (cur->child[!x] != NULL && cur->child[!x]->cnt > 0)
    {
        ans += (1ll << i);
        cur = cur->child[!x];
    }
    else
        cur = cur->child[x];
}
return ans;
}

ll xor_less_equal_k(ll n, ll k) // returns number of trie
    elements t_i such that (n ^ t_i) <= k
{
    ll ans = 0;
    node *cur = root;
    for (ll i = MX - 1; i >= 0; i--)
    {
        if (cur == NULL)
            break;
        ll x = (n & (1ll << i)) ? 1 : 0;
        ll z = (k & (1ll << i)) ? 1 : 0;
        if (x == 1)
        {
            if (z == 1) // 1 ^ y <= 1
            {
                if (cur->child[1])
                    ans += cur->child[1]->cnt;
                cur = cur->child[0];
            }
            else // 1 ^ y <= 0
                cur = cur->child[1];
        }
        else
        {
            if (z == 1) // 0 ^ y <= 1
            {
                if (cur->child[0])
                    ans += cur->child[0]->cnt;
                cur = cur->child[1];
            }
            else // 0 ^ y <= 0
                cur = cur->child[0];
        }
    }
}

```

```

    }
}
if (cur) // (n ^ t_i) == k
    ans += cur->cnt;
return ans;
}
};

// version alphabet [a-z]

// usage:
// trie t;

struct trie
{
    struct node
    {
        ll cnt; // number of strings ending in the subtree(node)
                    including this node
        vector<string> wend;
        node *child[26];

        node()
        {
            for (ll i = 0; i < 26; i++)
                child[i] = NULL;
            cnt = 0;
        }
    };

    node *root;

    trie()
    {
        root = new node;
    }

    void insert(string s)
    {
        node *cur = root;
        for (ll i = 0; i < (ll) s.size(); i++)
        {
            cur->cnt++;
            ll x = s[i] - 'a';
            if (cur->child[x] == NULL)
                cur->child[x] = new node;
            cur = cur->child[x];
        }
        cur->cnt++;
    }
}

```

```

        cur->wend.push_back(s);
    }

    ll dfs(node *cur) // counts number of nodes other than
        root
    {
        if (cur == NULL)
            return 0ll;
        ll ans = (cur != root);
        for (ll i = 0; i < 26; i++)
            ans += dfs(cur->child[i]);
        return ans;
    }
};

```

7.6 z function

// <https://cp-algorithms.com/string/z-function.html>

// Note:
 // z[i] is the length of the longest common prefix between s
 // and the suffix of s starting at i
 // z[0] = 0 (not well defined)
 // [l, r] is the known right most range that has matched
 // with prefix of s
 // Each iteration of the while loop will increase the right
 // border r of the match segment.
 // Since r can't be more than n - 1, this means that the
 // inner while loop won't make more than n - 1 iterations

```

vector<ll> z_function(string s)
{
    ll n = s.size();
    vector<ll> z(n);
    z[0] = 0;

    for (ll i = 1, l = 0, r = 0; i < n; i++)
    {
        if (i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }

    return z;
}

```